GENESIS 2.5: ~~optimisation~~ CUDA and ~~opt-in~~ OpenCL ~~/CUDA~~ acceleration for existing models of the GENESIS ~~/PGENESIS~~ compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a]*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b]*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator. ~~We release~~ GENESIS 2.5 ~~, adding opt-in OpenCL and CUDA backends for the Hodgkin–Huxley channel-update path and ,~~ adds optional CUDA and OpenCL backends to its `hsolve` solver, including a GPU Hines ~~tree-elimination kernel for dendritic trees, leaving every existing model, script and MPI workflow unchanged. On cluster hardware the accelerator reaches 57× end-to-end speedup on a~~ elimination for dendritic-tree populations, without changes to existing models or scripts. On 160 ~~000-compartment population, matching the fp64 CPU solver to 10^{-7}~~000 compartments over 10 ~~V. Measured against other simulators on one node, it runs~~ 000 steps an NVIDIA A100 runs 24.0× faster end to end than the ~~Vogels–Abbott COBAHH benchmark 2.3× faster than CoreNEURON on a single CPU core, and on the GPU it overtakes the GPU-native Arbor beyond 18–64 ms of simulated time depending on the card; CoreNEURON's GPU stays 1.23× ahead on that spiking benchmark. GENESIS can now run a spiking network on the GPU, though not faster than on the CPU . Its solver handled one spiking cell at a time, so a network needed one solverper cell, 4000 of them for the benchmark used here, and that cost dominated. One solver now holds a whole layer, which by itself makes the CPU arm 1.4× faster. On the GPU the network is no quicker~~CPU solver. In double precision it matches the CPU solver to ten significant figures. Spiking networks with zero axonal

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl` (Grzegorz Marcin Wójcik)

delay run slower, because the ~~cells exchange spikes every timestep and all 100 000 steps run as separate GPU calls~~host delivers spikes every step. The release also makes ~~$O(n^2)$~~ model construction linear ~~and repairs seven defects, three of them in the solver~~ in the common case and fixes three Hines-solver defects of GENESIS 2.4~~ships~~.

Keywords

GENESIS ; PGENESIS ; ~~OpenCL ; CUDA ; GPU acceleration ;~~ compartmental neural simulation ; Hines solver
; GPU ; CUDA ; OpenCL

Required Metadata

Current code version

Main text

1. Motivation and significance

   ~~Compartmental simulators remain essential where channel kinetics and dendritic geometry cannot be abstracted away.~~ GENESIS [1] ~~is~~ and its MPI version PGENESIS are among the longest-maintained ~~simulators in this class, with PGENESIS extending it to MPI; both are still used in production workflows, scripted in SLI , independent of newer ecosystems.~~

   ~~Modern workstations ship capable integrated GPUs, but GENESIS has no accelerator backend: every channel-kinetics update runs~~ compartmental simulators. Models are written as SLI scripts, cell files and channel prototypes, often accumulated over many years, and the compute engine is `hsolve`, an implicit Hines solver [3] that compiles a model into a flat program at setup. Until this release every step of that program ran on the CPU~~whatever hardware is present. A GENESISor PGENESIS user has no path to accelerator support that does not begin with abandoning their models and scripts.~~ .

   ~~**Existing accelerated simulators, and what they ask of a user.**~~ ~~Compartmental simulation on GPUs is not new, and the alternatives are mature. NEURON [13] is the most widely used simulator in this class; CoreNEURON [14] is its optimised compute engine, which strips the interpreter and runs the same models on GPUs through OpenACC, reaching large speedups on multi-rank and multi-GPU systems. Arbor [16] was written from scratch for many-core and GPU~~

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.6.2
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/WarsawIQ/genesis-2.5` (release v2.6.2, archived at `https://doi.org/10.5281/zenodo.23451767`)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions); see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, CUDA C++, OpenCL C, MPI (PGENESIS), Python and POSIX shell (reproduction and analysis)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, flex, bison, ncurses. Optional: CUDA 12.x (tested 12.8 on NVIDIA A40 and A100), an OpenCL 1.2 runtime (tested NVIDIA, AMD ROCm 6.3.1, Mesa rusticl 26.1), MPICH or Open MPI for PGENESIS
C7	If available Link to developer documentation/manual	`README.md`, `reproduce/README.md`, `paper/docs/REPLICATION.md`
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

~~hardware, with a performance-portable back end and its own model description. NEST [17] and Brian 2 [18], the latter reaching GPUs through GeNN [19], simulate networks of point neurons rather than dendritic morphologies. They are therefore not in~~ **Relationship to GENESIS 2.4.** GENESIS 2.5 is an independently maintained derivative of GENESIS 2.4, released under the same licence as a fork of the project's repository (`genesis-sim/genesis-2.4`, last changed in 2019). It is not an official release of the GENESIS project. Its

number continues GENESIS 2. GENESIS 3 took a different path, a modularisation of the simulator into stand-alone components based on the Neurospaces project [7, 6, 8, 9, 10]; GENESIS 2.5 keeps the code base of GENESIS 2.4 and runs its scripts unchanged. This paper describes release v2.6.2 of GENESIS 2.5; its numbers were measured on v2.6.0, whose computations v2.6.2 keeps (Section 3). The second author has used GENESIS and PGENESIS since his doctoral work [12] and met its maintainer, Dave Beeman, at the EU Advanced Course in Computational Neuroscience in Óbidos in 2001; the first author completed his PhD under the second author's supervision in 2023. On 30 September 2026 we offered the work to the original project: the ~~comparison below: NEURON, CoreNEURON and Arbor are the simulators a GENESIS user could move a compartmental model to~~fixes as a pull request, and the ~~only ones that would be running the same computation~~whole release as its next version. Dave Beeman had not replied by the time we submitted this revision; the solver and construction fixes are open as pull request #7 in that repository.

~~Each of these is a strong choice~~ **Alternatives.** NEURON [13] with its engine CoreNEURON [14] and Arbor [16] run compartmental models on GPUs and are strong choices for a new model. ~~None is a path for~~ For an existing GENESIS model ~~. CoreNEURON accelerates NEURON's models, not GENESIS's; Arbor and Brian 2 require the model to be re-expressed in their own descriptions; and a GENESIS user's investment is typically in SLI scripts, `.p` cell files and channel prototypes accumulated over years. Porting is not a mechanical translation — as this work found, two implementations of the same published benchmark differ enough in channel kinetics and connectivity construction that establishing their equivalence~~ each means re-expressing it in another description and showing that the port computes the same thing, which is itself experimental work~~(~~. For the published network of Section 3 ~~). The gap~~ it took a transcription of the channels and a comparison of spike trains. NEST [17] and Brian 2 [18] simulate networks of point neurons, and GeNN [19] generates GPU code for such networks. GENESIS 2.5 ~~addresses is therefore not "fastest simulator" but "acceleration without migration": the same models and scripts , unchanged, on the hardware the user already has~~runs existing GENESIS scripts on a GPU without porting them.

Contributions. ~~This work contributes:~~

(a) ~~Opt-in OpenCL and CUDA backends for the~~ CUDA and OpenCL

backends for `hsolve`~~channel-update path, behind one entry point, leaving every existing CPU, MPI, model and SLI-script workflow unchanged~~, selected at run time, in single or double precision, falling back to the CPU for anything they cannot compute.

(b) `hines_tree_eliminate`, a GPU ~~tridiagonal (Hines [3]) elimination kernel extending acceleration from isopotential cells to axially-coupled dendritic trees, running the identical elimination the CPU solver does~~Hines elimination for populations of dendritic trees.

(c) ~~Seven correctness fixes to released code, none of which requires a user to change a model. Three are in~~ Three fixes to the Hines solver of GENESIS 2.4~~distributes and reach every multi-neuron `hsolve`, accelerator or not: `inject`-driven voltage transients were silently suppressed for all but the first neuron. Four are in the accelerator as first released, where a model using synaptic channels, a `spike` element, GHK or concentration pools could be dispatched to the GPU and returned wrong voltages with no error raised; such models are now detected at `SETUP` and computed on the CPU.~~, which with several cells under one solver computed wrong voltages (backward Euler) or diverged and crashed (Crank–Nicolson), and with one compartment returned a wrong backward-Euler voltage under `chanmode 2`; and model construction in linear time. Both apply with or without a GPU and are open as a pull request to the original project.

(d) ~~A latent $O(n^2)$ model-construction bottleneck, unrelated to accelerator support, reduced to linear~~ A comparison with NEURON, CoreNEURON and Arbor in which every arm is measured in one session, and a reproduction package that regenerates every number in this paper from raw data (Section 3).

(e) ~~A head-to-head evaluation against NEURON, CoreNEURON and Arbor, on a published network model and on a multi-compartment population, with the competing simulators' GPU backends included and the equivalence of the independent implementations verified rather than assumed.~~

~~Why a GPU helps here is specific to the workload. In a Hodgkin–Huxley model the per-compartment gating integration dominates the cost and is embarrassingly parallel: each compartment advances from its own voltage and its own table lookups, uncoupled from every other within the step. The state is small and the arithmetic intensity low, so the work maps onto thousands of lightweight threads.~~

That makes the channel-update path the natural first target, ahead of the tridiagonal solve.

We provide both backends because they buy different things. OpenCL is vendor-neutral and runs on the integrated GPUs shipped with ordinary workstations, including devices without double precision, which is why the kernel is fp32 throughout. CUDA targets the datacenter cards where most cluster time is available and the profiling tools are standard. Both sit behind one entry point, so the choice is a build flag and neither the model nor its SLI script changes.

We name this release "2.5" rather than "3.0" deliberately: "GENESIS 3" [2] produced independent successor lines (Neurospaces/Heccer [8], MOOSE)evolving into the CBI federation [6, 7, 10], not a single artifact comparable to GENESIS 2.4. This release is not a re-architecture.

2. Software description
   (a) Software architecture

   GENESIS 2.5 leaves the existing architecture (

   The backends sit behind one entry point in `hsolve` (Fig. 1). The SLI interpreter, `hsolve` solver, PGENESIS MPI partitioning) unmodified and adds one component: an OpenCL backend invoked from `hsolve` when an attached element uses `chanmode=4` (or 5) with real ion-channel state. Two dispatch modes are implemented:

   (b) *Per-step dispatch* (`ocl_chip_update`): one `clEnqueueNDRangeKernel`call per simulation step, with channel state uploaded and downloaded every step.

   (c) *Multiloop dispatch* (, via): a single kernel dispatch runs all K steps in an inner GPU-side loop, amortising the per-step PCIe transfer cost that otherwise dominates wall time the element tree, the scheduler and the PGENESIS MPI layer are those of GENESIS 2.4. A user selects a backend with `chanmode` and environment variables. Nothing in a model script changes.

   An accepted model runs on one of three paths. *Per step*: the device updates channels and solves each step, and the host delivers synaptic events and spikes between steps. Networks whose cells exchange spikes take this path; on it a spike reaches some of its targets one step later than on the CPU (Section 3). Every accelerator run emits a non-empty profiling summary, which guards against two silent failures: an `hsolve` with no attached channels never reaching the kernel, and a kernel that fails to build falling back to CPU unnoticed. The kernel is fp32 throughout, so it runs on runtimes without double-precision

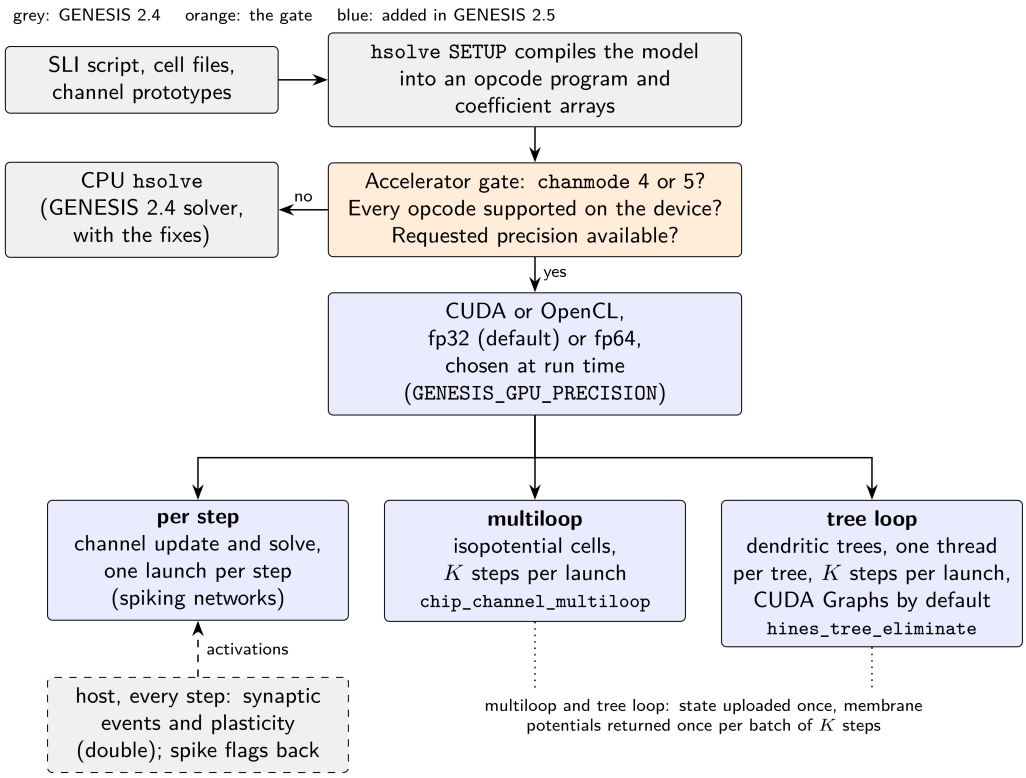


Figure 1: How a model reaches the GPU. `hsolve` compiles the model into an opcode program at `SETUP`. A gate checks the program against what the kernels implement; anything else stays on the CPU. Accepted models run as one of three dispatches: per step, batched over many steps for isopotential cells (`chip_channel_multiloop`), or batched for dendritic trees (`hines_tree_eliminate`). Synaptic events are delivered by the host every step.

~~support; the rest of `hsolve` stays `double`, converting only at the buffer boundary.~~

~~Both dispatch modes update each compartment independently, exact only for single-compartment (isopotential) neurons: a real dendritic tree's compartments are coupled through axial resistance, solved on the CPU by `hsolve` as a compiled bytecode program performing a bottom-up/top-down Gaussian elimination over the tree's tridiagonal system once per step. We add a third kernel, (; CUDA port in), executing this identical bytecode on the GPU, one thread per disconnected tree(neuron), operating on the same flat opcode /coefficient arrays the CPU solver already builds at `SETUP` time. Because independent trees never reference each other's rows (the combined program is block-diagonal),~~

~~threads need no synchronisation. The channel and elimination kernels dispatch back to back on the same queue/stream, so the~~ (d)). *Multiloop*: for isopotential cells without synaptic input, one launch runs K ~~-step batch still uploads and downloads state once. /select this path whenever the `hsolve` contains a multi-compartment tree, leaving the original kernel for single-compartment networks.~~ steps. *Tree loop*: for populations of dendritic trees, the channel update and `hines_tree_eliminate` run back to back for K steps. On the two batched paths the state goes to the device once and the membrane potentials come back once per batch, so a run of K steps costs one transfer instead of K.

~~Synaptically coupled spiking networks reach the kernels in this release.~~ **Design decisions.**

i. *The kernels interpret the solver's own program.* Both backends execute the opcode program `hsolve` ~~tracked one spike generator per solver, so such a network had to be built as one solver per cell; it now tracks one per compartment, and a whole layer becomes a single solver whose single-compartment solve the channel kernel completes on the device. One boundary remains, and it is a scheduling one rather than a kernel one: a network with zero axonal delay exchanges spikes every step, so multiloop batching does not apply and each step pays a dispatch. Section~~ already builds, so every model it accepts runs unchanged. GPU-native simulators generate code per mechanism instead. Interpreting costs divergence and a fixed setup time (Section 3~~measures what that costs.~~

ii. ~~Software functionalities~~

iii. ~~Unmodified serial (`genesis`), headless (`nxgenesis`), and MPI (PGENESIS) execution paths.~~

iv. ~~OpenCL channel-update kernel for Hodgkin Huxley-style `tabchannel` gating (Na X^3Y^1, K X^4), dispatched per-step or batched via multiloop.~~

v. ~~A CUDA backend at the same call site (): a line-for-line fp32 port behind an identical entry point, so one model and harness drive either backend.~~

vi. ~~Device-side dispatch confirmation distinguishing a genuine GPU run from CPU fallback.~~

vii. ~~A driven-spiking benchmark ()exercising the repaired `inject` path .)~~; in exchange, existing models need no porting.

viii. ~~Three corrected Hines-solver defects (forward-elimination fast-path guard, backward-elimination root handling, /row selection), independent of GPU support.~~ *The tree solve needs no synchronisation.* The elimination program of a population is block-diagonal: each tree touches only its own rows. Each thread therefore runs the CPU's elimination for one tree, with no barrier. The only change needed was to split, per tree, the opcode that joins two trees in the CPU program.

ix. ~~, a GPU tridiagonal elimination kernel in both backends, extending acceleration to axially-coupled dendritic trees (Section 3).~~ *What the kernels cannot compute is refused, not approximated.* Release v2.5, which the first version of this paper described, accepted models with synaptic, spike, GHK and concentration elements and returned plausible but wrong voltages. Such programs are now detected at `SETUP` and computed on the CPU.

x. ~~Five corrected $O(n^2)$ model-construction defects, restoring linear-time construction independent of GPU support (Section 3).~~ *The device owns the state it computes.* With synaptic input the host writes only the slots it owns each step. Release v2.5 uploaded the whole state, which overwrote gating variables the kernel had just updated.

~~The tridiagonal kernel decides whether any of this reaches realistic morphologies. Accelerating channel updates alone leaves the axial coupling on the CPU, where the solve becomes the bottleneck as soon as the channel work is taken out of it~~ **Execution model.** Four properties of the device code determine when the GPU pays off. *Work imbalance.* `hines_tree_eliminate` runs one thread per tree, so the threads of a warp wait for the largest tree among them. A population that mixes trees of 8 and 64 compartments in every warp takes 1.08× as long per step as uniform trees with the same total number of compartments. The CPU pays about the same for this mix (1.09×), so the waiting costs little. Grouping the same trees by size is slower (1.27×, E7), because it puts all the long trees on a few multiprocessors, which then finish last. *Divergence.* Compartments with different channel sets follow different opcode streams through the interpreter, so heterogeneous cells cost more than their compartment count suggests. *Transfers.* On the batched paths transfers are fixed per batch, not per step (above). On the per-step path they are paid every step. *Launch overhead.* In the tree loop on the A100 the host needs 6.2 µs

per step to launch the two kernels, which run for 76 µs (Nsight Systems, $N = 10\,000$ trees). ~~Moving it~~ The launches overlap the kernels and do not limit the loop. CUDA Graphs, which replay a recorded sequence of launches, cut the host's part to 0.5 µs per step. In a run of $K = 50\,000$ steps they save $3.0 \pm 1.0\%$ of the run time on the A100 and $1.9 \pm 0.5\%$ on the A40 (E8). At $K = 5000$ they save $4.8 \pm 2.3\%$ on the A100 and $0.7 \pm 1.2\%$ on the A40, which is within the noise. Graphs change no result, so they are on by default for the tree loop. On the spiking network they make no difference within error ($-2.4 \pm 3.9\%$). There the cost is the host's event delivery and the synchronisation it forces every step. Moving spike delivery to the device is ~~what lifts acceleration from isopotential point-neuron networks to populations of real dendritic trees, the regime Section 3 measures~~the next step for that workload.

(d) ~~Illustrative examples~~

~~Every measurement below comes from one of two UMCS "Lunar" cluster nodes: inf02, a Xeon Gold 6342 with an NVIDIA A40, and inf03, a Xeon Platinum 8358 with an A100. CPUarms are single-threaded unless stated.~~

~~**Reproducing the GPU multiloop result.** builds N driven Hodgkin Huxley neurons under one `hsolve` (`chanmode=4`) ; K dispatches all K steps in one GPU call. Both arms are wall-clocked identically at $K = 50\,000$ on the A40 and A100.~~

~~Table ?? reports both backends after the scheduler and solver fixes; speedup grows with N throughout.~~ Software functionalities

~~**Numerical fidelity of the fp32 accelerator.** compares the classic 1952 squid AP across CPU~~ **Precision.** Single precision is the default. `GENESIS_GPU_PRECISION=fp64` selects double precision at run time in the same binary, and a device without it refuses with a message and computes on the CPU. In fp64 ~~, GPU per-step, and GPU multiloop paths:~~ the GPU reproduces the CPU solver to every printed digit (ten significant figures) on our verification models; in fp32 ~~agrees with fp64 to 2×10^{-7}~~it agrees to within 2×10^{-7} ~~V over a single AP, all N neurons identical to $< 10^{-6}$~~V over one action potential on every device we tested. Synaptic weights and plasticity are computed by the synapse objects on the host in double precision in every mode. Only the resulting activation reaches the device. Section 3 measures the cost of double precision on each card and compares the

two precisions over a 10 V (Fig. ??). Over long spiking runs, fp32/fp64 traces drift in spike *phase* but preserve waveform and firing rate, why timing uses driven neurons while correctness uses a single-AP windows run. Runs longer than 10 s, with or without plasticity, and PGENESIS runs in which several MPI ranks share one GPU have not been tested.

Absolute difference between the accelerator's fp32 membrane voltages and the CPU's fp64 ones, for one driven spiking neuron over 6400 steps. Both dispatch modes are drawn — one kernel call per step, and one call covering a batch of them — and they overlap.

CUDA backend on NVIDIA hardwarePortability. The CUDA backend builds from the same source (`make USE_CUDA=1`) against the identical harness, and reproduces the fp64 CPU action potential to 7×10^{-8} V, confirming parity with both the CPU and OpenCLpaths. That sweep is 10 replicates on each of the two UMCS cluster GPUs. The two cards are close at every size, and the OpenCL backend trails CUDA by roughly 8% — about what a faithful port of the same kernels costs, and the price of running on hardware CUDA does not reach.

These are *end-to-end* speedups, and every figure in this paper uses that one definition. Timing the two arms by different clocks inflates the ratio tenfold and can invent differences between cards; measured consistently the OpenCL backend runs the same kernels on any OpenCL 1.2 device. We test it on the NVIDIA A40 and A100 are indistinguishable ($21.0 \pm 0.3\times$ vs. $21.9 \pm 0.3\times$).

At $N = 500$ the accelerator barely pays for itself: fixed costs dominate a run the CPU finishes in under a second.

What each accelerator backend is worth against the CPU solver, and how that changes with population size. Single-compartment multiloop kernel (`hh_spiking_benchmark.g`, $K = 50\,000$ steps), fp32 GPU against fp64 CPU on the UMCS cluster. Both arms of every ratio are wall-clocked around the whole process, so construction, transfers and per-step host work sit inside each figure. CUDA is the mean of 10 replicates with propagated uncertainties, OpenCL of 3. Raw data: .

Multi-compartment GPU acceleration. The multiloop kernel above is exact only for single-compartment (isopotential) neurons: for a real dendritic tree it silently drops all axial coupling. (Section 2.1) performs this elimination on-device, dispatched whenever the `hsolve` contains a multi-compartment

~~tree.~~

~~*Correctness.* Linear-chain and branching benchmarks compare GPU against~~ A100 and on the AMD Radeon 890M integrated GPU under two independent implementations, AMD ROCm and Mesa rusticl (E9). On the same chip the two differ: ROCm exposes double precision and rusticl does not, so under rusticl a request for fp64 ~~CPU across `NCOMP` $= 1$–32 and up to $N = 10\,000$ neurons: agreement is 10^{-7} V, matching single-compartment fidelity. One topology is not handled: a single-compartment branch off a branch point, which violates the kernel bootstrap's seed-row assumption. Both backends detect it at initialisation and compute that `hsolve`~~ is refused and the model computes on the CPU~~, so it costs speed rather than accuracy. A separate platform limit: on~~ . On an integrated GPU that also drives the display, ~~the~~ a long tree dispatch trips the driver's ~~reset above ∼22~~watchdog. For that reason the OpenCL backend declines the batched tree dispatch above 20 000 compartments ~~. The same configuration runs on the A40, so `GENESIS_OCL_TREE_MAX_NCOMPTS` caps model size there and is 0 on dedicated hardware.~~

~~*Speed.* Table ?? and Fig. ?? report speedup (mean ± std, 10 replicates) on the two UMCS cards, sweeping $N = 100$–$50\,000$ at `NCOMP` $= 16$, a range unreachable before the model-construction fix (Impact):~~ by default, prints a message and runs the model step by step; on a dedicated card `GENESIS_OCL_TREE_MAX_NCOMPTS=0` lifts the limit, as in all our measurements, and E9 checks that the limit is enforced. Under rusticl, which submits to the graphics queue, the same kind of limit stops the longest single-compartment run: the driver resets the GPU during the network of $N = 50\,000$ ~~took 23.8 ± 0.3~~cells batched over 50 ~~s to build there, versus not finishing before. Branching topology gives statistically indistinguishable speedups and is omitted from the figure.~~ 000 steps. Run one dispatch per step, the model takes 47.5 s, slower than the laptop's CPU at 32.2 s, while ROCm runs it batched in 5.7 s. Release v2.6.1 splits such a run into dispatches of about half a second, with the same result, so under rusticl it too runs batched on the GPU.

~~Two figures per card answer different questions. The *step-phase* speedup times the simulation loop alone and measures what the kernel can do: both cards are slower than~~

(e) Sample code

A model needs no change as long as its solver uses `chanmode 4` or `chanmode 5`, modes GENESIS 2.4 already has. An existing script

that builds its solver as

```
create hsolve /net/solver
setfield /net/solver path "/net/##[][TYPE=compartment]" \
    chanmode 4
call /net/solver SETUP
```

runs on the GPU when started with `nxgenesis` (CUDA) or `nxgenesis_ocl` (OpenCL) from the release. Batching and precision are chosen when the run starts, for example

```
GENESIS_CUDA_MULTILOOP=<K> GENESIS_GPU_PRECISION=fp64 \
    nxgenesis -batch model.g
```

The start-up lines name the backend, the precision and the dispatch path in use, and a model the device cannot compute is reported and run on the CPU.

3. Illustrative examples

Every number in this section was measured on release v2.6.0 on the UMCS "Lunar" cluster (inf02: Xeon Gold 6342 and NVIDIA A40; inf03: Xeon Platinum 8358 and NVIDIA A100). Release v2.6.2, which the metadata cite, fixes all the defects found during the work on this paper and computes the same results: on both cards its fp32 output is byte-identical to that of v2.6.0, and its fp64 output agrees with the CPU solver to 10^{-10} V. CPU arms are single-threaded. Times are wall-clock times of the whole process, and each value is a mean over replicates run in interleaved order. All runs, CPU ~~at $N \le 500$, pull ahead past $N \approx 500$–1000, and reach $37.3 \pm 0.1\times$ (~~arms included, were bound to the GPU's NUMA node, so both arms of a speedup used the same socket. A speedup is the ratio of two means; its uncertainty combines the two arms' relative sample standard deviations in quadrature. On inf02 the A40~~) and $80.8 \pm 1.0\times$ (A100) at $N = 50\,000$, neither plateaued. The *end-to-end* speedup wall-clocks the whole process, which is what a user waits for: it is far lower, $3.62 \pm 0.05\times$ (~~'s socket is the slower one: a CPU arm takes 1.19× as long there as on the other socket, so against a CPU arm on the other socket the A40~~) and $3.90 \pm 0.07\times$ (A100) at $N = 50\,000$. The gap is not a kernel deficiency but Amdahl's law applied to a short run — at $K = 200$ steps the unaccelerated construction phase dominates, and the end-to-end figure climbs toward the step-phase ceiling as K grows. Quoting only the step-phase number would overstate what the accelerator delivers in practice. ~~'s speedups would be that much smaller. On inf03 the GPU's socket is slightly faster (0.99×). The CPU arms use the same `hsolve` mode as the GPU arms (`chanmode 4`).

On the trees `chanmode 3` is faster still by 1.08×, so against it the tree speedups would be smaller by that factor, while `chanmode 1`, our CPU reference before this revision, is 4.2× slower with the same voltages. ~~The cards nearly coincide end-to-end despite the A100 leading by more than 2× on the step phase: the GPU phases take 7.83 s (A40) against 8.06 s (A100) at $N = 50\,000$. Raw data: .~~

(a) Speed on populations of cells

Table 2 gives the speedups for the single-compartment network, and Fig.

~~*Parallelisation granularity.* assigns one GPU thread per tree, so speedup comes from spreading many neurons across threads, not from parallelising elimination *within* a tree. For a single detailed morphology alone ($N = 1$), only one thread is active: measured directly, the kernel is ~20 21~~ 2 those for the dendritic trees, end to end and for the step phase alone. The end-to-end gain grows with run length as the fixed setup cost is amortised (Fig. 3). On the longest run of Fig. 3 the speedup is 17.1× ~~*slower* than the CPU at `NCOMP` = 2000 and 8000, stable with tree size as expected for a single-thread bottleneck. The kernel therefore accelerates population-scale simulation (Table ??), not the single-detailed-cell regime.~~ on the A40 and 24.0× on the A100. Double precision takes 1.50× the fp32 time on the A40 and 1.17× on the A100.

~~What the tree-elimination kernel is worth on dendritic trees, and how much of it survives into the whole run. Linear chains at `NCOMP` = 16, batched multiloop ($K = 200$, $K = 100$ for $N \geq 5000$), mean ± std over 10 replicates. *Step-phase* times the simulation loop alone and measures the kernel; *end-to-end* wall-clocks the whole process including model construction, both arms timed identically (). The end-to-end runs build the population with one `createmap`, as network models do. " " = not measured. Raw data: .~~

	NVIDIA A40		NVIDIA A100	
N	CUDA	OpenCL	CUDA	OpenCL
500	1.51 ± 0.03	1.49 ± 0.02	1.62 ± 0.05	1.77 ± 0.09
5 000	8.29 ± 0.08	8.31 ± 0.10	8.65 ± 0.31	9.16 ± 0.49
50 000	19.28 ± 0.20	19.33 ± 0.21	20.60 ± 0.24	20.64 ± 0.17

Table 2: Single-compartment network, $K = 50\,000$ steps: speedup of each backend over the CPU solver (E3, 10 replicates).

~~**Population-scale model construction was $O(n^2)$; now $O(n)$.** Pushing toward the ~31 000-neuron Blue Brain Project scale [24] exposed a defect independent of GPU support: 527 000~~

Speedup of the tree-elimination kernel over the CPU against population size, 16 compartments per neuron, log-log, mean ± std over 10 replicates. Colour is the card; solid lines time the simulation loop alone, dashed ones the whole run including model construction, at $K = 200$ steps.

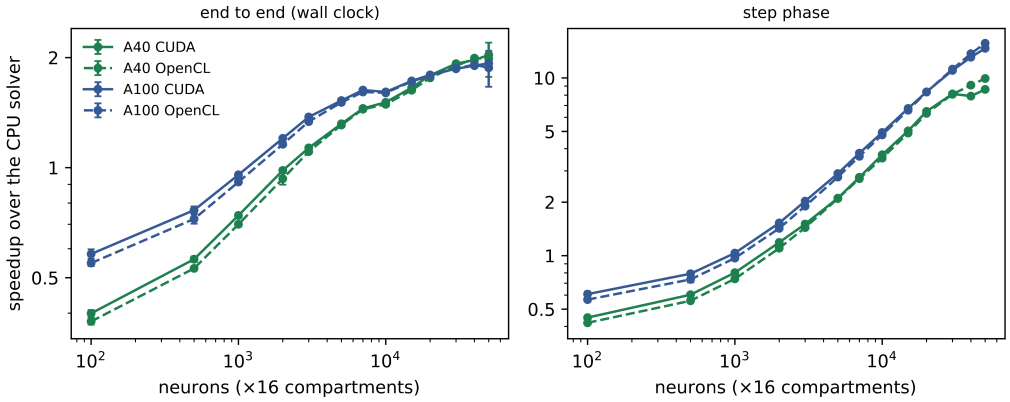


Figure 2: Dendritic trees, $N \times 16$ compartments, 200 steps: speedup over the CPU solver, end to end and step phase, on the A40 and A100 (E3).

compartments did not finish building within 900 s. Profiling gave a fitted exponent of 2.31, traced to five pre-existing linear-scan-per-element patterns in element creation, path resolution and `hsolve` SETUP, none GPU-specific — each rescanning siblings, paths or compartments where hashing or amortised growth would do. Fixing all five (Fig. **??**) gives an exponent of 0.99: that population now builds in 14.6 ± 0.2 s, and 1.7 million compartments in 51.5 ± 0.6 s. Output is byte-identical on every validation script, confirming a pure performance fix; full diagnosis in .

The multi-compartment figures are a floor set by the benchmark, not a ceiling set by the kernel. The sweeps above use $K = 200$ ($K = 100$ for $N \geq 5000$), which at $dt = 50\,\mu$s is 5–10 ms of simulated activity, short enough that fixed costs dominate both arms. Repeating the measurement at $N = 10\,000$ over a range of K (Table **??**) separates them: end-to-end speedup rises from 4.3× at $K = 200$ to 57.0 ± 0.9× at $K = 5000$, and step-phase from 13.7× to 116.2×. Both grow because the one-time costs they contain are amortised over more steps: model construction for the end-to-end figure, buffer setup and the single batched dispatch for the step-phase figure.

Fitting fixed and marginal costs separates them: construction

Model-construction wall time vs. N, log-log, before (single replicate) and after (mean ± std, 3 replicates) the five $O(n^2)$ fixes, extended to $N = 100\,000$. The "before" series stops at $N = 8000$: larger N did not finish within the tested budget.

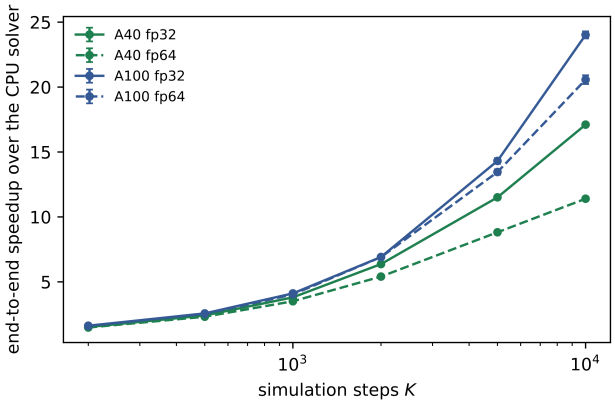


Figure 3: End-to-end speedup against run length, $10\,000 \times 16$ compartments, fp32 and fp64, both cards (E2).

takes ≈1.3 s on both arms, while one step costs 2.31×10^{-2} s on the CPU against 1.42×10^{-4} s on the GPU, a ratio of 163×. At $K = 5000$ the GPU still reproduces the CPU voltages to 10^{-7} V, so this is amortisation, not skipped work. Table **??** reports the short-K figures because they are what the standing benchmark measures, but a 250 ms simulation of this population already reaches 57× end-to-end.

How much of the speedup you actually get depends on how long you run: the same model at four run lengths. $N = 10\,000$ neurons × 16 compartments on the UMCS A40, mean ± std over 5 replicates, both arms timed identically. Each arm carries about 1.3 s of model construction whatever K is. Re-measured after the corrections of Fig. **??** the same figure comes out 55.4× against the 57.0× here, the difference sitting in the CPU arm; the campaign quoted is the fuller 5-replicate one. Raw data: .

Reproducing the PGENESIS scaling result. The existing MPI path is unaffected: a strong-scaling sweep ($N = 2400$ HH neurons, $P = 1$–24 ranks)gives $E_P = 0.95$ at $P = 2$, an efficiency knee near $P \approx 5$, and 3.1–4.0× at $P = 4$–8.

Head-to-head against NEURON and CoreNEURON. The speedups above are internal to GENESIS, so they say nothing about other simulators. We therefore ran the Vogels Abbott COBAHH network [21, 22] in GENESIS ()and in the reference

NEURON implementation (ModelDB 83319, its directory) on the same cluster node (Table ??, Fig. ??), with every CPU arm single-threaded. GENESIS 2.5 completes the 5.0 s simulation in 46.9 ± 2.1 s as published, and in 33.2 ± 0.7 s built as one solver per layer, which this release allows for the first time: 2.30× faster than CoreNEURON on the same core, 2.88× faster than NEURON 9.0.2.

CoreNEURON exists to make NEURON models faster, discarding the interpreter and rebuilding the data layout for vectorisation.It is in turn 1.25× faster than NEURON 9.0.2, the expected ordering, which suggests that arm is configured correctly.

Because these are independent implementations, we verified their equivalence rather than assuming it: every arm has the same cell count, the same 64 excitatory and 16 inhibitory inputs per cell, the same 50 ms of external drive, and settles into the same 27 Hz regime.

(b) Against other simulators

The stock benchmark cannot run under CoreNEURON at all, since its channels are ChannelBuilder objects rebuilt inside the interpreter. We transcribed them into NMODL, reproducing the published COBAHH rates, after which CoreNEURON yields a spike train byte-identical to NEURON's. The same mechanisms, compiled into an Arbor catalogue, give the fourth arm: 149.4 ± 1.1 s on the A100 that runs CoreNEURON in 27.0Table 3 lists every arm of the spiking-network comparison, measured in one session on one node in interleaved order, so no ratio in it mixes machines or days. Arbor cannot express the benchmark's zero axonal delay— it advances in minimum-delay epochs — so that arm uses one timestep , so its arm uses a delay of one step (50 µs). Arbor advances in epochs of half the shortest connection delay and exchanges spikes every step, which is also why our own batched dispatch is unavailable here. Every cross-simulator arm runs on inf03, since the two nodes' CPUs differ and times taken on both would not be comparable. The GPU arms are measured on both cards once per epoch [20]; with this delay it does so at least once per step, the same per-step exchange that keeps GENESIS from gaining on the GPU. We did not profile Arbor to confirm that this is where its time goes.

On the dendritic trees GENESIS pays more than Arbor at setup and less per step. In single precision it is slower than Arbor on short runs and faster beyond 48 ms of simulated time on the A100

and 13 ms on the A40; at the longest run it takes 0.74× Arbor's time on the A100 and 0.55× on the A40 (Fig. 4). Harness: .

CoreNEURON on the GPU. Leaving CoreNEURON on the CPU , where it was never meant to stay, would flatter us. Its OpenACC backend would not coexist with a working NEURON, so we ran it standalone: `nrncore_write` writes the model out and `special-core` simulates it. On one In double precision, which Arbor always uses, GENESIS is slower at every run length we measured, taking 1.10× Arbor's time on the A100 CoreNEURON completes the same network in 27.0 ± 0.1 s — 2.83× faster than its own CPU arm, and 1.23× faster than our best CPU time and 1.05× on the A40 at the longest run. Its spike count differs from the CPU arms by 6% (592 865

We profiled GENESIS and CoreNEURON on one CPU core (E6). GENESIS is faster on both workloads, 2.22× on the network and 2.56× on the trees. GENESIS reads its channel gates from tables; CoreNEURON evaluates exponentials, and the exponential function alone takes 47% of its time on the network. On the network GENESIS spends 38% of its time on synapses, spike events and the calls that reach them, against CoreNEURON's 12%. This host-side work runs on the CPU every step, so the GPU cannot speed it up. On the trees the channels take 61% of GENESIS's time and the linear solve 21%, against 558 824) and from GENESIS by 10% : CoreNEURON's GPU path requires cell permutation, which reorders synaptic delivery, and the network is chaotic, so counts diverge while the regime does not. This is a throughput comparison in a common regime, not a claim of identical trajectories.

This is the boundary of what the release offers. Built as one solver per layer the model does run on the accelerator, but only at parity with its own CPU arm (Section 5), so 43% and 17% of CoreNEURON's GPU stays ahead by 1.23×. Where the accelerator pays — multi-compartment populations — the comparison below is GPU on bothsides. Most of the rest is model construction in both, plus idle library threads in CoreNEURON.

Do the three integrate the same model? Measured rather than assumed (Fig. ??): NEURON and Arbor agree to 0.02 mV at the action-potential peak and to 0.01% on firing rate, while GENESIS peaks 4.6 mV lower and fires faster, its Hodgkin–Huxley rates being written about −70 mV where NEURON's use −65. Setting the comparison up corrected three errors in our own

Simulator	Hardware	Wall time (s)	Relative
GENESIS 2.5, one solver per cell (as published)	CPU	46.9 ± 1.0	1.36
GENESIS 2.5, one solver per layer	CPU	34.6 ± 0.9	1
GENESIS 2.5, one solver per layer, fp32	A100	39.6 ± 0.6	1.14
GENESIS 2.5, one solver per layer, fp64	A100	40.3 ± 0.5	1.17
NEURON 9.0.2, NMODL channels	CPU	100.1 ± 0.6	2.90
NEURON 9.0.2, ChannelBuilder channels	CPU	128.4 ± 1.5	3.71
CoreNEURON 9.0.2	CPU	76.5 ± 0.2	2.21
CoreNEURON 9.0.2	A100	27.1 ± 0.1	0.79
Arbor 0.10.0	A100	150.0 ± 1.8	4.34

Table 3: ~~Where GENESIS 2.5 stands against the other simulators on a published spiking network, GENESIS in bold.~~ Vogels–Abbott COBAHH network [21, 22], 4000 cells, ~~80 synapses each, $dt = 0.05$ ms, 5.0~~5 s~~simulated, one UMCS node; CPU arms single-threaded. Every~~ : wall time of every arm ~~settles into the same activity regime~~in one session on inf03 (E1, ~~26.8–30.1 Hz~~5 replicates, ~~so the wall times cover comparable work~~interleaved). ~~Means ± std~~ Relative: wall time over ~~3 replicates, except the ChannelBuilder row, a single run~~ that of ~~the benchmark in its original form. The~~ GENESIS ~~GPU arm comes from a later session~~ with one solver per layer on the ~~same node, where its own~~ CPU~~reference ran 34.9 ± 0.4 s~~; ~~that pair~~ below 1 is ~~the one to divide, giving 1.04×, rather than rows taken months apart. Harness:~~ faster.

~~harness — Arbor's injection site and its reversal potentials, and a conductance scaled by the soma's area in GENESIS — none of which moved a wall time by more than 1.3%. Timing is unaffected otherwise: with no synapsesand no events, switching the injection off moves no arm by 2%.~~

~~**GPU against a GPU-native simulator.** In the comparison above the accelerator reaches only parity with its own CPU arm, because a zero-delay spiking networkdispatches every step (Section 2.1). To test it where batching does apply we used a model — $N = 10\,000$ neurons, each a chain of 16 compartments, HH channels, current injection, no synapses — reimplemented in NEURON and in Arbor [16], which was written from scratch for GPU hardware.~~

(c) When to use the GPU

~~Which simulator is faster depends on run length, and the crossover~~

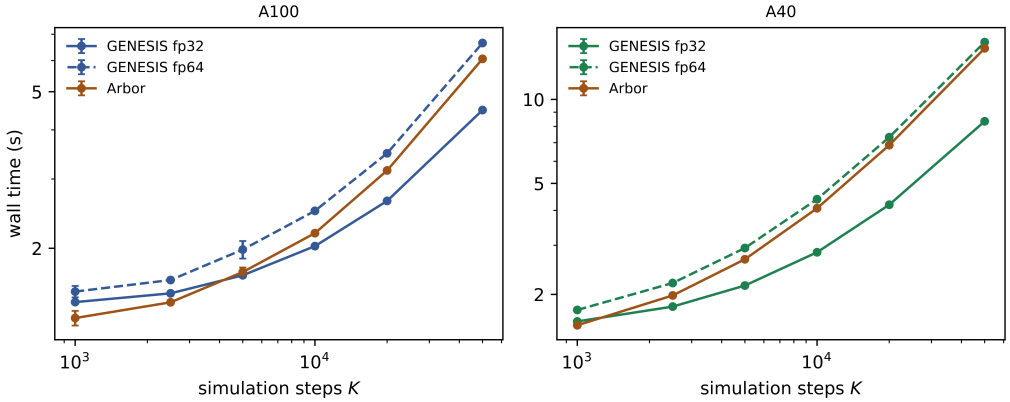


Figure 4: ~~The measurements of Table ??, drawn. Green is~~ GENESIS ~~, grey~~ and Arbor on the ~~simulators it is compared~~ same GPU against ~~; each label names its device~~run length, ~~and the dashed guide marks the fastest~~ GENESIS ~~time. Means of 3 replicates, except the ChannelBuilder row~~in fp32 and fp64 (E4).

~~is measurable. Both simulators are linear in K (Table ??). Fitted over six run lengths, our marginal cost is 33% lower while Arbor starts 0.20 s ahead, so the lines cross at $K \approx 6400$ (Fig. 4). Below it Arbor wins on setup; above it, where modelling studies sit, GENESIS 2.5 is faster. The A40 gives the same ordering with a wider margin, crossing at $K \approx 1800$: our kernel is fp32 where Arbor computes in double, and that card's double-precision throughput is a fraction of the A100's . A comparison at one run length therefore reports whichever side of the crossing its author chose. The device profile locates the cost: our step phase exceeds kernel time by 0.21 s at both runlengths, so it is one-time dispatch setup, not per-step work. This is the cost of the design that keeps existing models working: Arbor generates specialised code per mechanism, while our kernels interpret the `hsolve` opcode program at run time. On single-threaded CPU the GENESIS solver is 2.02× slower than CoreNEURON here,~~ Table 4 gathers what the measurements above say about where the ~~opposite of Table ??, because the two benchmarks stress different things: the spiking network is dominated by synaptic event handling over single compartments, this one by the tridiagonal solve, where CoreNEURON's vectorised layout is stronger.~~
accelerator pays off and where it does not.
~~Wall-clock time against run length for the multi-compartment benchmark, one panel per card, $N = 10\,000$ neurons × 16~~

~~The same multi-compartment model in three simulators, on CPU and on GPU, at two run lengths. $N = 10\,000$ neurons $\times$ 16 compartments, $dt = 0.01$ ms, UMCS node inf03, mean $\pm$ std over 3 replicates, both arms of every pair timed identically. Thread counts differ and are stated: Arbor uses all cores by default, the others are single-threaded. The CPU arms appear only at $K = 5000$, where they were measured; extrapolating them would be an estimate. The GPU rows come from the sweep Fig. 4 fits, so table and figure cannot disagree. The models match in geometry, passive properties, conductances, injection and timestep (Fig. ??); this is a throughput comparison, not a claim of identical trajectories. Harness: .~~

Workload	GPU?	Why
One detailed morphology, or fewer than about 3000 trees	No	too few trees to fill the device; below the threshold the GPU is slower (Fig. 2)
Population of dendritic trees	Yes, from about 3000 cells (200 steps)	many trees fill the device (Fig. 2)
Isopotential cells without synapses	Yes	21$\times$ at $N = 50\,000$ (Table 2)
Spiking network, zero axonal delay	No: 1.14$\times$ slower	spikes are exchanged every step, so every step is a dispatch (Table 3)
Long runs	Gain grows with length	the fixed setup cost is amortised (Fig. 3)

Table 4: Where acceleration helps.

~~compartments, mean $\pm$ std over 3 replicates. The dotted line and diamond mark where the fitted lines cross.~~

(d) Correctness over a long run

~~One cell's first action potential in three simulators, at the soma and at the far end of the dendrite, which moves only if axial coupling is integrated. Each simulator is drawn twice, one backend wide and pale with the other dashed on top, so a coinciding pair reads as agreement rather than a missing line. Data: .~~ The GPU and CPU solvers do not order spikes the same way. On the GPU a spike emitted in one step reaches its targets in the next; the CPU solver delivers it within the step to the cells after the sender in its loop. With this network's zero axonal delay the two schedules differ by one step (50 µs) for some synapses, and the network is chaotic, so it amplifies the difference. Over 10 s of the spiking

network, twice the length of any comparison above, the GPU's spike train first departs from the CPU's after 2.65 ms, in fp64 and in fp32 alike. The two precisions part at the same step, so the cause is the order, not rounding. The CPU is just as sensitive to its own order: with one solver per cell instead of one per layer, the same network and seed depart from the first CPU run after 2.35 ms. From then on only statistics can be compared. Against the CPU's 27.2 Hz, the spike count differs by +0.9% in fp64, -0.3% in fp32 and -0.7% for the CPU in the other order; the KS distances between the interspike-interval distributions are 0.002, 0.003 and 0.006. Four other seeds, other draws of the same network, fire at 24.6–29.2 Hz with KS distances of 0.021–0.049. Each run repeats spike for spike on its own backend. So the GPU's spike count differs from the CPU's about as much as the CPU's own count changes under another update order. Its interspike intervals are closer to the CPU's than that run's are, and both differences are far smaller than the change from one draw of the network to another.

~~A reproduction pack is included: re-measures the claims here~~

(e) Reproducing these results

Each number in this paper has a row in `reproduce/claims.csv` that names its data file, the rows and formula it comes from, and the script that measures it; a number taken from the literature names its source. `make_numbers.py` computes the values in the text from those files, and a lint lists any number typed by hand. `sh reproduce/run_all.sh` runs the stages that measured them on the reader's ~~own hardware , regenerates the figures from those measurements, and prints each published value beside theirs. Two rows need hardware or a toolchain it cannot assume — CoreNEURON on a GPU,~~ hardware and computes the same numbers from the new data with the same code, beside the published ones: about 14 minutes on an A100, build included, for Table 2, the run-length speedup and CUDA Graphs, and about 70 minutes at the paper's full sizes (NVIDIA GPU, CUDA 12.x). Run so from a fresh clone, the 10 s simulations of Section 3(d) gave back the published spike trains spike for spike. With `--cpu-only` it measures the results that need no GPU in about three minutes on any Linux machine, and the ~~A40 arm — and are documented instead. Raw data and full steps: .~~ same path runs in a container (`docker build -t genesis25-cpu . && docker run --rm genesis25-cpu`) with nothing else

installed, as in our earlier pipeline for GENESIS experiments [23] . Both run in the repository's CI on every commit. A generated table in the README says which results need which hardware. The comparison simulators and the reference model are built by committed recipes with pinned versions and checksums. Rebuilding them from an empty directory reproduced the published CoreNEURON GPU run to the spike.

4. Impact

   Two of the release's changes help every GENESIS user, with or without a GPU. The first is the solver fixes. In GENESIS 2.4, several cells under one `hsolve` silently gave wrong voltages under backward Euler: the first cell's were perturbed and the others were never computed. Under Crank–Nicolson the solve diverged or crashed. Networks are often built this way, so published results may be affected.

   GENESIS~~/PGENESIS users with channel-kinetics-heavy models now have a path to GPU acceleration (Section 3) without migrating models, SLI scripts, or MPI workflows — the barrier an architecturally different successor such as MOOSE would impose. The accelerator's scope is deliberate: both kernels interpret the `hsolve` opcode program directly and implement the five opcodes a current-driven `tabchannel` model produces, so compartments needing anything else (synaptic channels, a `spike` element, GHK, concentration pools) are detected at `SETUP` and computed on the CPU. That boundary defines what this releaseaccelerates, and a published network model is what established where it lies (Section 1).~~

   ~~The correctness and construction fixes reach further than the accelerator does. The three Hines-solver defects silently suppressed `inject`-driven voltage transients for every neuron but the first in any multi-neuron `hsolve`, and the five linear-scan patterns behind $O(n^2)$ construction (Section 3) blocked population-scale models outright. Both predate this release and are fixed for every GENESIS/PGENESIS user, whether or not an accelerator is available.~~

5. ~~Conclusions~~

   GENESIS2.5 ~~adds opt-in OpenCL and CUDA acceleration to GENESIS 2.4/PGENESIS 2.4 for Hodgkin Huxley `tabchannel` models, without changes to model files, SLI scripts, or MPI workflows (Section 3 reports speedup and fidelity on cluster-class hardware). Beyond the single-compartment multiloop kernel, extends acceleration to axially-coupled multi-compartment trees at population scale; the single-detailed-cell regime is not helped by this design, which we measure and report.~~

The release also carries seven correctness fixes to code already in users' hands. Three are Hines-solver defects that silently suppressed `inject`-driven voltage transients in any multi-neuron `hsolve`, and benefit any GENESIS/PGENESIS user, accelerator or not; four are in the accelerator as first released computes each cell exactly as if it were alone, which a committed test checks against a single-cell reference. A third defect stored, for a cell of one compartment under `chanmode` 2, the backward-Euler solution minus the previous voltage in place of the voltage. The fixes and their tests are open as a pull request to the original project (Section 1). Separately, five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction time are fixed, restoring linear scaling and making population-scale models buildable at all.

Measured against other simulators on a published network model , GENESIS 2.5 is 2.30× faster than CoreNEURON and 2.88× faster than NEURON 9.0.2 for single-threaded CPU execution of the Vogels–Abbott COBAHH benchmark, after verifying that the two implementations are equivalent (Section 3). That benchmark is a spiking network, which this release accelerates for the first time though only to parity with its own CPU arm, so CoreNEURON on an A100 still runs it 1.23× faster. On multi-compartment populations, where the accelerator does apply, GENESIS 2.5 overtakes the GPU-native Arbor beyond 64The second is construction. Building a model took time quadratic in its size, growing as compartments$^{2.18}$: 272 ms of simulated time on an A100 000 compartments took 8 minutes to build, and 18 ms on an A40.

Future work. Three limits measured here say what to do next, in the order we think they matter to users.

Parallelism inside the tree. The kernel gives each tree a thread, so it speeds up populations and does nothing for a single detailed morphology: there only one thread runs, at that rate the 1.7-million-compartment population of Fig. 5 would take about 7 hours. Four scans of the element tree are now constant- or linear-time. Construction grows as compartments$^{1.02}$, and the GPU is some 20× slower than the CPU. Much of the modelling literature works on exactly that cell, and reaching it needs cyclic reduction or a similar scheme within each tree. This is where the design is furthest from finishedsame population builds in 94.9 s. This holds for models built element by element, the common case; copying a whole prototype cell thousands of times under one parent is still quadratic. That population has 100 000 cells of 17 compartments, about three times

the 31 000 neurons of a reconstructed neocortical microcircuit [24], though with far simpler cells. With the tree kernel, population studies such as parameter sweeps over thousands of morphological neurons can run on a workstation instead of in a cluster queue, in the simulator the models were written for. Table 4 lists the practical regimes, including where the GPU does not help.

~~*Cutting the dispatch.* Lifting the one-spike-generator limit is worth 1.41× on the CPU by itself, and~~ Two parts of this work apply beyond GENESIS. The first is the ~~device then reproduces the network's spike count to 1.1%~~refusal rule: a backend that cannot compute a model should say so and leave it to the CPU. Release v2.5 approximated such models and returned plausible but wrong voltages. The second is the reproduction mechanism, a map from every published number to its raw data that regenerates the text and runs in CI. It does not ~~make it faster. A zero-delay network exchanges spikes every step, so nothing can be batched and each step pays a dispatch, which costs more than the kernel saves: the GPU arm is 1.04× slower than its own CPU. A bigger model does not rescue it — at 12 500 cells the two arms are level — because the accelerable fraction sits near 59% whatever the size, holding the ceiling at 2.4× that dispatch then spends. The work to do is cutting dispatch, not building larger networks.~~ depend on GENESIS and would serve any computational paper.

~~*Precision over long runs.* fp32 is what lets the kernels run on integrated GPUs with no double-precision hardware, and it costs ~10^{-7} V against the fp64 CPU over the runs measured here. Whether it stays that small over the far longer runs plasticity studies need is untested.~~

~~CoreNEURON's multi-rank MPI configuration is still unmeasured and would place GENESIS~~

6. Conclusions

GENESIS 2.5 ~~more completely. Packaging into a container-based deployment pipeline [23] is in progress~~runs existing GENESIS models on a GPU with no change to their SLI scripts, cell files or MPI workflows. The CUDA and OpenCL kernels execute `hsolve`'s own compiled program, in single or double precision chosen at run time. Populations of dendritic trees gain the most, up to 24.0× end to end. Single detailed morphologies and zero-delay spiking networks run slower on the GPU: the first has too few trees to fill the device, and the second needs host-side event delivery every step. For spiking networks the next step is spike delivery on the device. The solver and construction fixes help every GENESIS user and are open as a pull request to the original project. Every measured number in

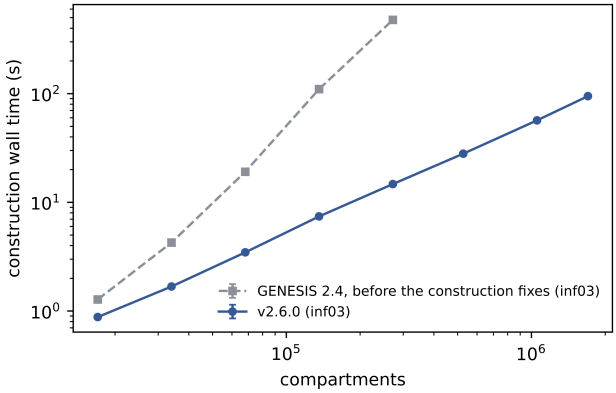


Figure 5: Model construction time against model size on inf03, 3 replicates each: GENESIS 2.4 before the construction fixes (the upstream branch without them, built on the node and timed the same way) and v2.6.0 (E3c). The code from before the fixes was measured up to 16 000 cells, where one build already takes 8 minutes.

this paper can be regenerated from raw data in the repository, the hardware-independent ones on any Linux machine or in a container.

CRediT authorship contribution statement

Karol Chlasta: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review and editing. **Grzegorz Marcin Wójcik**: Resources, Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

~~During the preparation of this work the~~ The authors used Anthropic's Claude to draft and edit the ~~manuscript and to assist~~ text and to help with the software and its ~~benchmarking. After using this tool, the authors~~ benchmarks. They reviewed and edited ~~the content as needed~~ all of it and take full responsibility for the ~~content of the~~ published article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acknowledgements

The authors thank ~~the~~ Maria Curie-Skłodowska University (UMCS) in Lublin and ~~the LubMAN UMCS~~ its LubMAN computing centre for access to the "Lunar" cluster~~(NVIDIA A100 and A40 GPU nodes) at the UMCS Institute of Mathematics, commissioned in 2015 [25]and since upgraded with the GPU nodes used in this work, and~~ [25]; Karol S. Karpowicz and Prof. Przemysław Stpiczyński for ~~provisioning and support of~~ setting up and supporting the compute environment~~on which the CUDA backend was built and validated. The authors also thank~~ ; and WarsawIQ for the AMD Radeon 890M ~~integrated GPU~~ on which the OpenCL backend was developed.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System*. Springer, 1998.

[2] ~~GENESIS 3 Development Plans.~~

[3] Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69–76. doi:10.1016/0020-7101(84)90008-4.

[4]

[5] Cornelis H, ~~Coop AD, Rodriguez AL, Beeman D~~Lu H, Esquivel A, Bower JM. ~~GENESIS 3.0 functionality enhanced by interface to multiple types of database~~Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* ~~2011, 12(Suppl 1):P15~~2007, 8(Suppl 2):P3.

[6] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[7]

[8] Cornelis H, ~~Lu H, Esquivel A~~Coop AD, Rodriguez AL, Beeman D, Bower JM. ~~Modeling a single dendritic compartment using Neurospaces and GENESIS-3~~GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* ~~2007, 8(Suppl 2):P3~~2011, 12(Suppl 1):P15.

[9] Cornelis H, Rodriguez AL, Coop AD, Bower JM. Python as a Federation Tool for GENESIS 3.0. *PLoS ONE* 2012, 7(1):e29018. doi:10.1371/journal.pone.0029018.

[10] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

[11] ~~Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing~~

[12] G.M. Wójcik, W.A. Kamiński. Liquid state machine built of Hodgkin–Huxley neurons and pattern recognition. *Neurocomputing* 58–60 (~~PDP~~2004) 245–251.

[13] Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* ~~, 2019, pp. 274-282~~1997, 9(6):1179–1209. doi:~~10.1109~~10.1162/~~EMPDP.2019.8671560~~neco.1997.9.6.1179.

[14] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.

[15] ~~Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456-492. doi:10.1016/j.cell.2015.09.029.~~
~~Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69-76. doi:10.1016/0020-7101(84)90008-4.~~
~~Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* 1997~~

[16] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation

Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 9(6): 1179–12092019, pp. 274–282. doi:10.116210.1109/neco.1997.9.6.1179EMPDP.2019.8671560.

[17] Gewaltig M-O, Diesmann M. NEST (NEural Simulation Tool). *Scholarpedia* 2007, 2(4):1430. doi:10.4249/scholarpedia.1430.

[18] Stimberg M, Brette R, Goodman DFM. Brian 2, an intuitive and efficient neural simulator. *eLife* 2019, 8:e47314. doi:10.7554/eLife.47314.

[19] Yavuz E, Turner J, Nowotny T. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 2016, 6:18854. doi:10.1038/srep18854.

[20] Arbor documentation. Interconnectivity: epochs and the minimal delay. `https://docs.arbor-sim.org/en/stable/concepts/interconnectivity.html`, accessed 17 October 2026.

[21] Vogels TP, Abbott LF. Signal Propagation and Logic Gating in Networks of Integrate-and-Fire Neurons. *Journal of Neuroscience* 2005, 25(46):10786–10795. doi:10.1523/JNEUROSCI.3508-05.2005.

[22] Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, et al. Simulation of networks of spiking neurons: A review of tools and strategies. *Journal of Computational Neuroscience* 2007, 23(3):349–398. doi:10.1007/s10827-007-0038-6.

[23] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.

[24] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.

[25] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. `https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.chtm`, 2015.

Current executable software version

~~Not applicable; this release is distributed as source code only (see Current code version above).~~

Nr.	(Executable) software metadata description	Please fill in this column
S1	Current software version	v2.6.2
S2	Permanent link to executables of this version	`https://github.com/WarsawIQ/genesis-2.5/releases/tag/v2.6.2`
S3	Legal Software License	GNU General Public License v2 or later
S4	Computing platforms/Operating Systems	Linux x86_64; tested on AlmaLinux 8.10 with CUDA 12.8 (NVIDIA A40, A100) and on Ubuntu 24.04 (TUXEDO OS) with ROCm 6.3.1 and Mesa rusticl 26.1 (AMD Radeon 890M)
S5	Installation requirements & dependencies	glibc and ncurses; for the GPU binaries an NVIDIA driver for CUDA 12, or an OpenCL 1.2 runtime
S6	If available, link to user manual	`README.md` in the repository
S7	Support email for questions	karol@chlasta.pl

Table 5: Software metadata (optional)